

MERGE SORT: DIVIDE AND CONQUER

Link to practice handout

<https://bit.ly/Divide-and-Conquer-Practice>

Divide and Conquer Algorithms

Algorithm Approach:

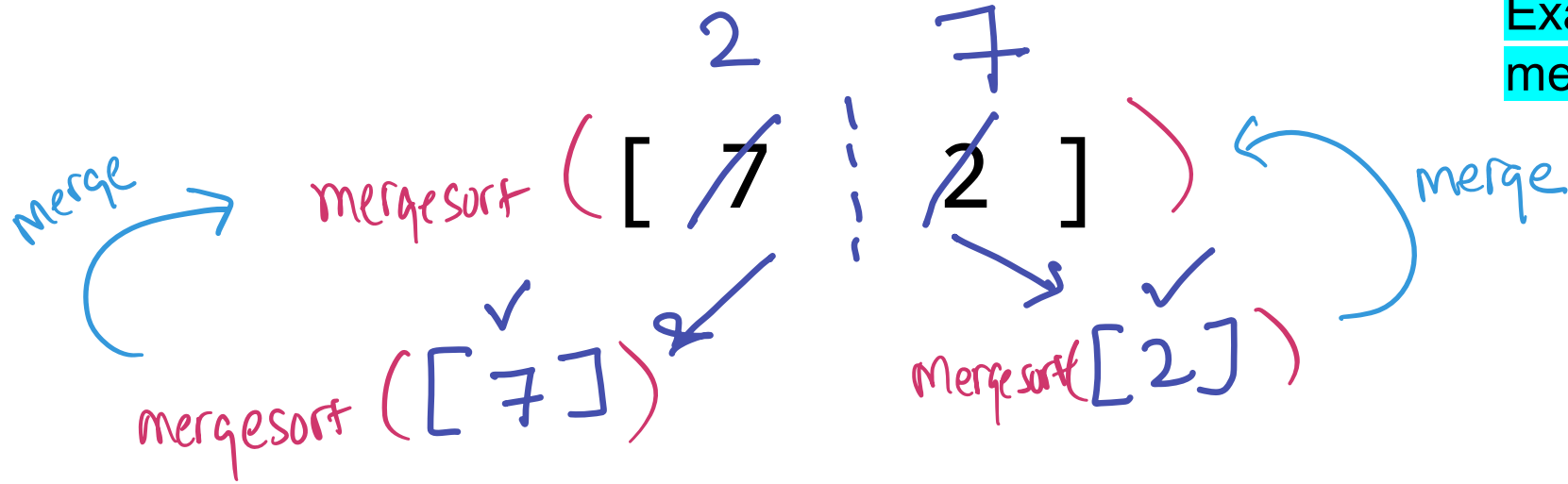
- **Divide** a large problem into sub-problems
- **Solve** each sub-problem
- **Combine** the solutions of sub-problems to obtain the solution for the original problem

Merge Sort Algorithm

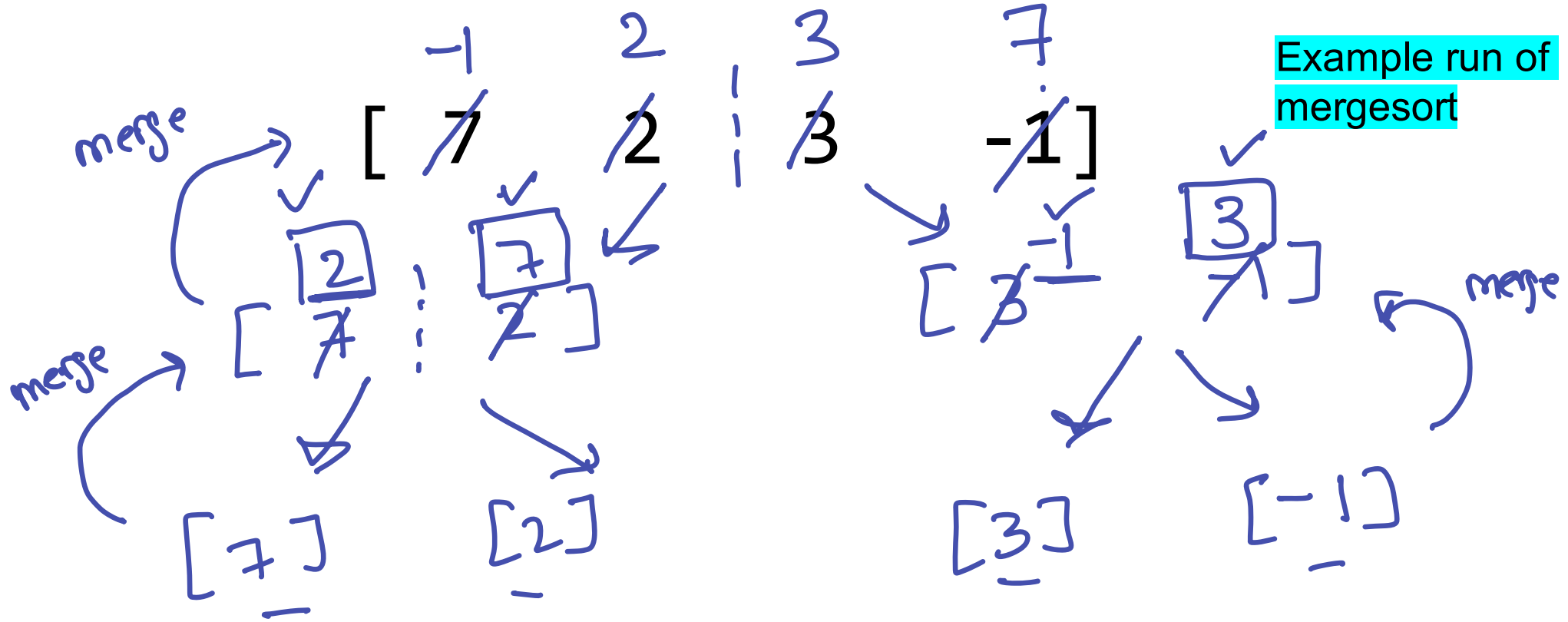
`MergeSort(vector v)`

- Divide `v` into left half and right half
- Sort the left half, then sort the right half
- Combine (merge) the two sorted halves

Example run of
mergesort



Example run of mergesort



[7 2 5 | 3 -1]

Example run of mergesort

[7 2 5]

[3 -1]

[7 2]

[5]

[3]

[-1]

[7]

[2]

What is the height of the binary tree trace of mergeSort?

- A. 1. B. 2 C. 3 D. 4 E. 5

Generalize the answer for an input vector of size n

Running Time Analysis

$[7 \quad 2 \quad 5 \quad 3 \quad -1]$ level 0
 $T(n) = \# \text{ copy operations to split lists} + \# \text{ comparisons to merge lists} + \# \text{ function calls}$

$[7 \quad 2 \quad 5] \quad [3 \quad -1]$ (n copies + n comparisons)
 level 1

$[7 \quad 2] \quad [5] \quad [3] \quad [-1]$
 level 2

$[7] \quad [2]$

level 3

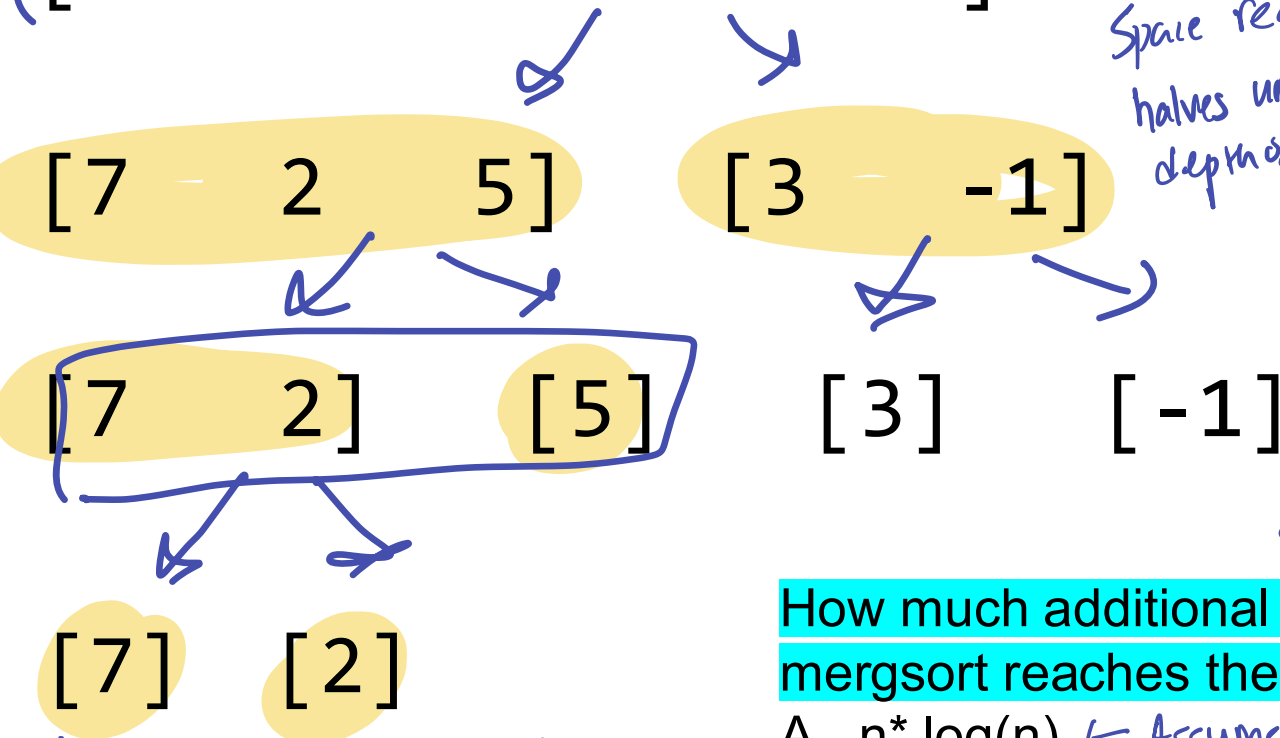
$T(n) = \log n \cdot (O(n) + O(n)) + 2n + 1$
 $\uparrow$ max depth of recursion $\uparrow$ copies per level $\uparrow$ comparisons per level
 bound on the number of function calls

Observe that at each level, mergesort needs to copy n elements to split lists and n comparisons.
 max number of levels, $H = \log n$

$$\begin{aligned}
 \# \text{ Function calls} &\leq 2^0 + 2^1 + \dots + 2^n \\
 &= 2^{n+1} - 1 \\
 &= 2 \cdot 2^{\log n} - 1 \\
 &= 2n - 1
 \end{aligned}$$

auxiliary $\rightarrow$ Space Analysis

$[7 \quad 2 \quad 5 \quad 3 \quad -1]$ $S(n) =$



Space required for splitting the input into left & right halves until maximum depth of recursion

$$= n + \frac{n}{2} + \frac{n}{4} + \dots + \frac{n}{n}$$

$$= n \left(1 + \frac{1}{2} + \frac{1}{4} + \dots + \frac{1}{n} \right)$$

converges to a constant

$\approx O(n)$

Space required for function calls until max depth of recursion

$$= O(\log n)$$

How much additional space is used by the time mergesort reaches the base case?

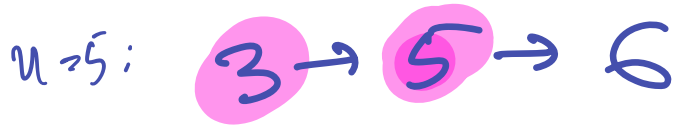
- A. $n * \log(n)$ $\leftarrow$ Assume all recursive calls are active simultaneously
- B. $n + n/2 + n/4 + n/8 + \dots + 1$
- ☒ C. $n + n/2 + n/4 + n/8 + \dots + 1 + \log(n) \approx O(n) + O(\log n) = O(n)$
- D. Something else

The main point to note is that the recursive function calls are not made simultaneously, so the space usage is not $O(n \log n)$

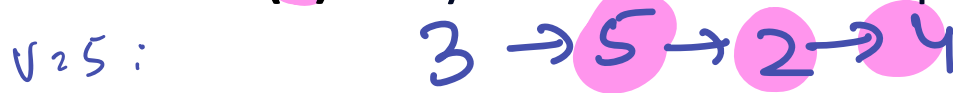
Path: a sequence of nodes in which each node is connected by an edge to the next.



Ancestor(u): any node that is on a path ending in u

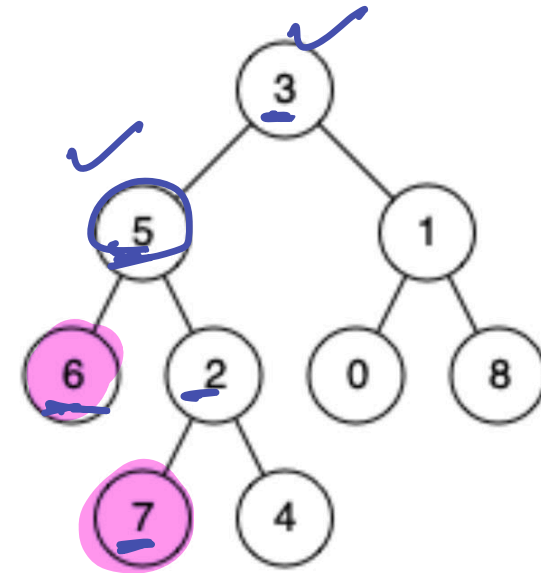


Descendant(v): any node that is on a path starting from v



Common ancestor(u, v): any node that is the ancestor of both u and v

Lowest Common ancestor(u, v): deepest node in the tree that is a common ancestor of u and v



Approach: Divide and Conquer

What is the LCA of each of the following?

5 and 1: 3

5 and 4: 5

6 and 7: 5

Base case:

if current node is either value or null return current node

Discuss how you would solve the problem with your neighbor, trace your solution, describe in words, then implement Leetcode or handout

<https://bit.ly/Divide-and-Conquer-Practice>

<https://leetcode.com/problems/lowest-common-ancestor-of-a-binary-tree/>

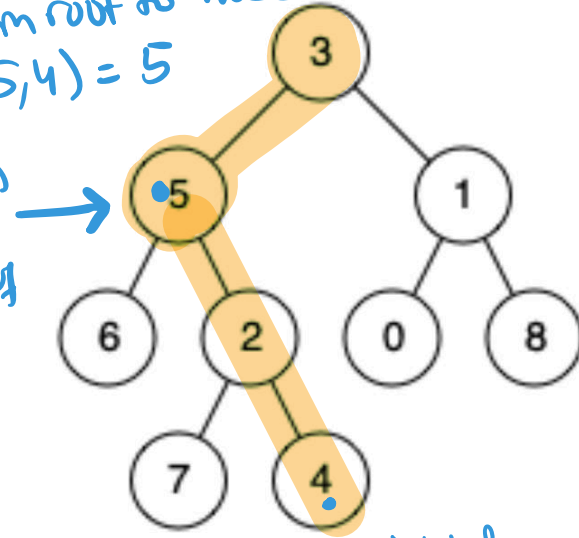
Finding LCA(5,4)

Path from root to node 5: $3 \rightarrow 5$

Path from root to node 4: $3 \rightarrow 5 \rightarrow 2 \rightarrow 4$

$LCA(5,4) = 5$

$LCA(5,4)$: deepest node in the tree that is common ancestor of 5 & 4



Outline:

Subproblems to solve find LCA & conquer

- Find LCA on left subtree
- Find LCA on right subtree
- Merge results to get LCA of root tree

consider base cases